Deliverable 2

Amalie K. Meltvedt and Maren J. Borgaas

Date: 30.10.2026

GitHub Repository Link

The GitHub repository can be accessed through this link:
https://github.com/marenjb/Deliverable-2.git

Importing the necessary libraries

In [1]:
```python
# Importing necessary libraries
from pathlib import Path
import re
import json
import numpy as np
import pandas as pd
```

Task 1

To create the file reader function, we used the `pathlib.Path` function to handle file paths. In order for it to work on both UTF-8 and ISO-8859-1/Latin-1 encoding, we used a try-except statement where it will first test UTF-8. If this results in a UnicodeDecodeError, the except block will run and test ISO-8859-1.

In [2]:
```python
# Using a try-except statement to create the file reader function
def read_file(file_path: Path) -> tuple[str,str]:
    """Read the files and return the text content and the encoding used"""
    try:
        file_text_content = file_path.read_text(encoding = "utf-8")
        encoding_used = "utf-8"
    except UnicodeDecodeError:
        file_text_content = file_path.read_text(encoding = "ISO-8859-1")
        encoding_used = "ISO-8859-1"

    return file_text_content, encoding_used
```

Next, we created a log parser function using the `re` module. The regular expression pattern is stored in the `pattern` variable, meaning the function will recognize lines that use the same pattern. `pattern.fullmatch()` and `if not match: ignore` ensures only exact matches are added, meaning noise lines will be ignored. It will then capture the key fields `timestamp`, `register_id`, `item`, `price`, `currency` and `status`, save them in dictionaries, and finally add the dictionaries in the `records` list.

In [3]:
```python
# Creating the log parser function
def parse_log(file_text_content: str) -> list[dict]:
    """Parse the log text and return the valid transaction records"""

    # Creating an empty list for the records
    records = []

    # Storing the regex pattern
    pattern = re.compile(r"\[(.*?)\] \[(.*?)\] item=(.*?) price=(.*?) (.*?) \((.

    # Going through each line and storing matching expressions in a match variab
    for line in file_text_content.splitlines(): # Splitlines to go through each
        match = pattern.fullmatch(line)

        if not match:
            continue

        # Collecting the key fields
        timestamp = match.group(1)
        register_id = match.group(2)
        item = match.group(3)
        price = match.group(4)
        currency = match.group(5)
        status = match.group(6)

        # Saving the key fields data in a dictionary
        record = {
            "timestamp": timestamp,
            "register_id": register_id,
            "item": item,
            "price": price,
            "currency": currency,
            "status": status,
        }

        # Appending the data to the records list
        records.append(record)

    return records
```

Task 2

To create a recursive directory search function that find all log files, we used `Path.iterdir()` to iterate through the items. We then used `.is_dir()` to check wether an item is a directory. If it is, the function will call itself recursively to search through the subdirectory. Because the recursive function call returns a list, we used `.extend()` to add each element from the returned list to `log_files` individually. If an item is a file, the code will check if it has a `.log` extension using `.suffix`, and then add it to the list.

In [4]:
```python
# Creating the log file finder function
def find_log_files(directory: str | Path) -> list[Path]:
    """Find all log files in a directory and in all subdirectories"""

    # Converting the directory to a Path object
```

```
    directory = Path(directory)

    # Creating an empty list to store the log files
    log_files = []

    # Iterating through the directory and its subdirectories
    for item in directory.iterdir():

        # Checking if the item is a directory, and using recursion if it is, add
        if item.is_dir():
            log_files.extend(find_log_files(item))

        # Checking if item is a file and if it has .log extension, adds it to th
        elif item.is_file() and item.suffix == ".log":
            log_files.append(item)

    return log_files
```

The function below is designed to traverse any combination of nested dictionaries and lists in `store_config.json`. The code uses `isinstance()` to check the datatype. For example, we first check whether data is a dictionary. We then use `.get()` to retrieve the values associated with the keys "active" and "id". If the register is active and its ID is a string, the ID is added to the set using `.add()`. To search through nested dictionaries and lists, the function uses recursion and `.update()` to add active registers to the set. A `set` will return only unique active register IDs.

In [5]:
```
# Creating the function to find active registers
def find_active_registers(data: object) -> set[str]:
    """Find all register IDs where active is true"""

    # Creating an empty set to store the active register IDs
    active_registers = set()

    # Checking if the data is a dictionary
    if isinstance(data, dict):

        # Checking if the register is active
        if data.get("active") is True:

            # Get the register ID
            register_id = data.get("id")

            # Checking if the register ID is a string
            if isinstance(register_id, str):
                active_registers.add(register_id)

        # Using recursion to go through the values of the dictionary and find ac
        for value in data.values():
            active_registers.update(find_active_registers(value))

    # Checking if the data is a list
    elif isinstance(data, list):

        # Using recursion to go through the items of the list and find active re
        for item in data:
            active_registers.update(find_active_registers(item))
```

```
    return active_registers
```

Task 3

The function `remove_price_outliers()` replaces price outliers with the median of the valid prices. First, we convert the input prices to a 1D numpy float array, so that we can do further calculations using numpy. We then use `np.mean()` and `np.std()` to find the mean and standard deviation. To find price outliers we use the formula given in the task description. We also ensure that values greater than 999, and 999 is defined as outliers. Then we define valid prices, which is prices that were not identified as outliers. This prevents the outliers to affect the replacing median value. Finally, we use this median to replace the outliers and return the cleaned array.

In [ ]:
```python
# Create a function to remove price outliers and replace them with the median pr
def remove_price_outliers(prices) -> np.ndarray:
    """Replace price outliers with the median price."""

    # Convert the input to a 1D numpy float array
    prices = np.asarray(prices, dtype=float).flatten()

    # Calculate the mean and standard deviation
    mean = np.mean(prices)
    std = np.std(prices)

    # Identify hardware glitch outliers like 999.00 or higher,
    # and statistical outliers (3 standard deviations from the mean)
    outliers = (prices >= 999) | (np.abs(prices - mean) > 3 * std)

    # Calculate the median of valid prices
    valid_prices = prices[~outliers]
    median = np.median(valid_prices)

    # Replace outliers with the median price
    prices[outliers] = median

    return prices
```

To create a function that computes the refund ratio, we first use type hints to specify that both `sales_count` and `refund_count` should be numpy arrays. When both inputs are numpy arrays it allows us to calculate the refund ratio for multiple observations using element-wise operations. We divide `refund_count` by (`sales_count` + 1e-6) to find the ratio, as given in the task descripton. We add 1e-6 to the denominator to prevent division by zero.

In [7]:
```python
# Create a function to compute the refund ratio
def compute_refund_ratio(
        sales_count: np.ndarray,
        refund_count: np.ndarray
) -> np.ndarray:
    """Computes the refund ratio for each element."""

    # Calculate the refund ratio element wise, handling division by zero
```

```
    refund_ratio = np.divide(
        refund_count,
        sales_count + 1e-6
    )  # Adding a small value to avoid division by zero

    return refund_ratio
```

Task 4

To convert the transaction dictionaries into a pandas dataframe, we used `pd.DataFrame()`. The `timestamp` column was converted to datetime with `pd.to_datetime()` and set as the DataFrame index. The DataFrame was then chronologically sorted using `sort_index()`.

In [8]:
```
# Converting to pandas dataframe
def transactions_to_dataframe(records: list[dict]) -> pd.DataFrame:
    """Convert transaction records to a chronologically sorted DataFrame"""

    # Converting the list of dictionaries into a df
    df = pd.DataFrame(records)

    # Converting the timestamp column to datetime
    df["timestamp"] = pd.to_datetime(df["timestamp"])

    # Set timestamp as the index
    df = df.set_index("timestamp")

    # Sort the dataframe chronologically by timestamp
    df = df.sort_index()

    return df
```

The dataframe created could then be used as input for another function that groups the data by item, resamples transactions in 1-hour windows, and calculates the total price. For this we converted the values in the `price` column to numeric values using `pd.to_numeric()`, then grouped them with `groupby()`. Because there are several currencies, we decided to group by currencies as well to ensure that prices in different currencies were not summed together. `resample("1h")` then divides the transactions into one-hour intervals, so that the total price for each item in each interval can be calculated with `sum()`. The result is returned in the `hourly_revenue` variable.

In [9]:
```
# Converting price column to numeric values and calculating the sums
def hourly_revenue_by_item(df: pd.DataFrame) -> pd.DataFrame:
    """Group transactions by item and calculate total price per hour"""

    # Convert price to numeric values
    df["price"] = pd.to_numeric(df["price"])

    # Group by item and currency and resample into one-hour windows
    hourly_revenue = (
        df.groupby(["item", "currency"])
        .resample("1h")["price"]
        .sum()
        .rename("revenue")
```

```
        .reset_index()
    )

    return hourly_revenue
```

Next, we used `pd.pivot_table()` to create a pivot table displaying total transaction revenue grouped by item across the status categories `SUCCESS`, `REFUND` and `CANCELLED`.

In [10]:
```
def revenue_pivot_table(df: pd.DataFrame) -> pd.DataFrame:
    """Create a pivot table of total revenue by item and transaction status."""

    # Convert price to numeric values
    df["price"] = pd.to_numeric(df["price"])

    # Create the pivot table
    pivot_table = pd.pivot_table(
        df,
        values="price",
        index=["item", "currency"],
        columns="status",
        aggfunc="sum",
        fill_value=0
    )

    return pivot_table.reset_index()
```

Finally we created a function that identifies the number of sales and refunds per item. First, it filters the transactions based on status, then groups them by item and counts the number of transactions per item. The counts are aligned using `add()` and `reindex()`, meaning items without refunds are also included and assigned a refund rate of 0. Next, the counts are converted to NumPy-arrays so they can be passed to the `compute_refund_ratio()` function we created in task 3. Finally, the calculated refund ratios are stored in a pandas Series, with each item as the index.

In [11]:
```
def refund_ratio_by_item(df: pd.DataFrame) -> pd.Series:
    """Calculate the refund ratio for each item."""

    # Count sales and refunds for each item
    sales_count = df[df["status"] == "SUCCESS"].groupby("item").size()
    refund_count = df[df["status"] == "REFUND"].groupby("item").size()

    # Align the counts so all items are included
    sales_count = sales_count.add(refund_count * 0, fill_value=0)
    refund_count = refund_count.reindex(sales_count.index, fill_value=0)

    # Calculate the refund ratio using the function from task 3
    refund_ratio = compute_refund_ratio(
        sales_count.to_numpy(),
        refund_count.to_numpy()
    )

    return pd.Series(refund_ratio, index=sales_count.index, name="refund_ratio")
```

Task 5

The different functions were then put together in a single workflow. This was achieved by creating a function that uses the `find_log_files` function created in Task 2 to find all log files in the data directory and read the `store_config.json` file. It then extracts the active register IDs using `find_active_registers()`, also defined in Task 2. Next, the function will create an empty `all_records` list. It then goes through each log file to read the text and identify the encoding using `read_file()` from Task 1. `parse_log()` extracts the valid transactions as dictionaries, adding them to the same `all_records` list. This list is converted to a dataframe, and `remove_price_outliers` is applied seperately to each currency to ensure that prices in different currencies are treated individually. Prices in NOK are multiplied by 10 to fix the system glitch described in the task scenario. Finally, the function generates the three reports (hourly revenue, revenue by item and transaction status, and refund ratios) in the correct CSV and JSON formats.

In [ ]:
```python
# Creating the main workflow function to run all the steps
def run_workflow(data_dir: Path) -> None:
    """Run the transaction-processing workflow and save all reports"""

    # 1. Find log files and extract active registers
    log_files = find_log_files(data_dir)

    with (data_dir / "store_config.json").open(encoding = "utf-8") as file:
        config_data = json.load(file)

    active_registers = find_active_registers(config_data)

    # 2. Ingest text logs and records into a list of dictionaries
    all_records = []

    for log_file in log_files:
        file_text_content, encoding_used = read_file(log_file)
        records = parse_log(file_text_content)
        all_records.extend(records)

    # 3. Create a Dataframe containing all parsed transactions
    df = transactions_to_dataframe(all_records)

    # 4. Clean price outliers separately for each currency
    df["price"] = df.groupby("currency")["price"].transform(
        # Lambda creates a small function that use the outlier removal on each c
        lambda prices: remove_price_outliers(prices.to_numpy())
    )

    # 5. Correct prices for NOK transactions
    nok_mask = df["currency"] == "NOK"
    df.loc[nok_mask, "price"] *= 10.0

    # Round prices to two decimal places
    df["price"] = df["price"].round(2)

    # 6. Generate resampled reports, pivot tables and refund ratios
    hourly_revenue = hourly_revenue_by_item(df)
    status_pivot_table = revenue_pivot_table(df)
    refund_ratio = refund_ratio_by_item(df)
```

```
    # 7. Save all required output files
    output_dir = data_dir / "output"
    output_dir.mkdir(parents=True, exist_ok=True)

    # Save cleaned and corrected transactions to CSV
    df.to_csv(output_dir / "cleaned_transactions.csv", index=True)

    # Save hourly revenue report to JSON
    hourly_revenue.to_json(
    output_dir / "hourly_revenue.json",
    orient="records",
    date_format="iso",
    indent=2
    )

    # Revenue grouped by item and transaction status
    status_pivot_table.to_csv(
    output_dir / "status_pivot_table.csv",
    index=False,
    float_format="%.2f"
    )

    # Refund ratio for each item
    refund_ratio.to_csv(
        output_dir / "item_refund_ratios.csv",
        float_format="%.2f"
    )
```

Finally the `run_workflow()` function can be applied to the data. The code below sets the data directory to `"data"` and calls the function to generate the final reports.

In [13]:
```
# Check whether the Python file is run directly or imported into another file
# This prevents the workflow from running unintentionally when imported
if __name__ == "__main__":
    data_dir = Path("data")
    run_workflow(data_dir)
```

This should create the `cleaned_transactions.csv`, `hourly_revenue.json`, `item_refund_ratios.csv` and `status_pivot_table.csv` files. To account for the different currencies, we have grouped by currency on all files except the refund ratios. To make it more organized we created an output folder to store the files in and limited the number of decimals to two.